ASSIGNMENT

Course: Full Stack Web Development

Course Code: 23CM4121

Assignment No.: 1 Date: 30-08-2026

Student Name: P. Govindamma Roll No.: A24126552104

1. TITLE

Implement Inheritance Types Using JavaScript

2. AIM

To implement and demonstrate the different types of inheritance in JavaScript — single, multilevel, hierarchical, multiple (via object mixins), and hybrid inheritance — using ES6 class syntax.

3. OBJECTIVE

- To understand the concept of inheritance in JavaScript using ES6 classes.
- To implement single inheritance using the extends keyword.
- To implement multilevel inheritance by chaining classes across multiple generations.
- To implement hierarchical inheritance where multiple child classes extend a common parent class.
- To implement multiple inheritance in JavaScript by mixing plain object methods into a class prototype using Object.assign(), since a class cannot directly extend more than one class.
- To implement hybrid inheritance by combining classical inheritance (extends) with an object mixin applied via Object.assign().
- To verify the behaviour of each inheritance type by creating objects and invoking inherited and own methods.

4. THEORY

Inheritance is an object-oriented programming concept that allows one class (child/derived class) to acquire the properties and methods of another class (parent/base class). In JavaScript, ES6 introduced the class and extends keywords, which provide a cleaner, class-based syntax over the underlying prototype-based inheritance model.

JavaScript supports the following types of inheritance:

- Single Inheritance — A single child class inherits from a single parent class using extends. Here, class Dog extends Animal inherits the eat() method (and the name set in the constructor) from Animal and adds its own bark() method.
- Multilevel Inheritance — A class inherits from a class which itself inherits from another class, forming a chain (Vehicle → Bike → Car). Each level adds its own method (move(), wheels(), windows()) while retaining access to all methods above it.

- Hierarchical Inheritance — Multiple child classes (Admin, Customer) independently extend the same parent class (User), each inheriting the login() method while adding their own distinct behaviour (deleteUser() / checkout()).
- Multiple Inheritance — JavaScript classes can extend only one class directly, so multiple inheritance is achieved here using plain object mixins: hall and bedRoom are ordinary objects whose methods are copied onto Area's prototype using Object.assign(Area.prototype, hall, bedRoom). This lets Area combine behaviour from two separate sources (sofa()/tv() and bed()/closet()) in addition to what it already inherits classically from Building.
- Hybrid Inheritance — A combination of more than one inheritance type in the same design: ToyCar classically extends Device (gaining turnOn() via extends), and is then mixed with the Battery object using Object.assign(ToyCar.prototype, Battery) to add charge(), combining classical inheritance with the mixin pattern.

In all cases, the child class gains access to the parent's (and, in the case of mixins, the mixed-in) methods without redefining them, which promotes code reuse and a clear object hierarchy.

5. ALGORITHM

- Create a JavaScript file to hold all five inheritance demonstrations.
- For single inheritance: define a parent class Animal with a constructor storing name and an eat() method, and a child class Dog that extends Animal and adds a bark() method. Create a Dog object and call both methods.
- For multilevel inheritance: define Vehicle → Bike → Car as a three-level chain, each adding one new method. Create a Car object and call all three inherited/own methods.
- For hierarchical inheritance: define a parent class User with a constructor storing username and a login() method, and two child classes Admin and Customer that both extend User independently. Create objects of both and call their methods.
- For multiple inheritance: define two plain objects, hall and bedRoom, each holding a couple of methods. Define a class Building with a constructor and house() method, and a child class Area that extends Building. Use Object.assign(Area.prototype, hall, bedRoom) to mix both objects' methods into Area, then create an Area object and call its house(), sofa() and bed() methods.
- For hybrid inheritance: define a class Device with a turnOn() method and a plain object Battery with a charge() method. Define ToyCar extends Device, then use Object.assign(ToyCar.prototype, Battery) to mix in the Battery methods. Create a ToyCar object and call both turnOn() and charge().
- Instantiate objects for each demonstration and call their inherited and own methods.
- Run the file using Node.js and observe the console output for each inheritance type.
- Verify the actual output against the expected output for each section.

6. PROGRAM

assignment.js

1.Single Inheritence:

```
class Vehicle {
```

```
    start() {
        console.log("Vehicle starts");
    }
}

class Car extends Vehicle {
    drive() {
        console.log("Car is driving");
    }
}

const car = new Car();

car.start();
car.drive();
```

2. Multilevel Inheritence:

```
class Person {
    introduce() {
        console.log("I am a person");
    }
}

class Student extends Person {
    study() {
        console.log("Student is studying");
    }
}

class CollegeStudent extends Student {
    attendClass() {
        console.log("College student is attending class");
    }
}

const student = new CollegeStudent();
```

```
student.introduce();
student.study();
student.attendClass();
```

3. Hierarchial Inheritence:

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  eat() {
    console.log(this.name + " is eating");
  }
}

// Child class 1
class Dog extends Animal {
  bark() {
    console.log(this.name + " is barking");
  }
}

// Child class 2
class Cat extends Animal {
  meow() {
    console.log(this.name + " is meowing");
  }
}

// Create Dog object
let dog = new Dog("Tommy");

console.log("Dog:");
dog.eat();
dog.bark();

console.log();
```

```
// Create Cat object
let cat = new Cat("Kitty");

console.log("Cat:");
cat.eat();
cat.meow();
4. Multiple Inheritence:
class Employee {
   work() {
      console.log("Employee is working");
   }
}

class Developer extends Employee {
   code() {
      console.log("Developer is writing code");
   }
}

class Manager extends Employee {
   manage() {
      console.log("Manager is managing the team");
   }
}

const developer = new Developer();
developer.work();
developer.code();

const manager = new Manager();
manager.work();
manager.manage();
```

7. CODE EXPLANATION

Code	Explanation
class ... extends ...	Creates a child class that inherits all properties and methods of the specified parent class.

class Dog extends Animal	Single inheritance: Dog inherits eat() from Animal (which stores name via its constructor) and adds its own bark() method.
Vehicle → Bike → Car	Multilevel inheritance: Car inherits from Bike, which itself inherits from Vehicle, forming a three-level chain (move() → wheels() → windows()).
Admin, Customer extends User	Hierarchical inheritance: two independent child classes inherit login() from the same parent class User, each adding its own method (deleteUser() / checkout()).
Object.assign(Area.prototype, hall, bedRoom)	Multiple inheritance via mixins: hall and bedRoom are plain objects holding methods; Object.assign copies their methods onto Area's prototype, so Area gains behaviour from two separate sources in addition to what it inherits from Building.
Object.assign(ToyCar.prototype, Battery)	Hybrid inheritance: ToyCar already inherits turnOn() classically from Device (extends); Object.assign then mixes in charge() from the Battery object, combining classical inheritance with the mixin pattern.
new ClassName()	Creates an instance (object) of the given class.
console.log()	Prints the result of each method call to the console for verification.

8. OUTPUT

Output: node assignment.js (Console)

```
Single Inheritance Buddy
is eating.
Buddy is barking.
Multilevel Inheritance
Moving forward
Bike has 2 Wheels
Car has 4 Windows
Hierarchial Inheritance
Anvitha logged in.
Sam logged in.
Anvitha deleted by admin
Sam checked out successfully.
Multiple Inheritance
Building House No: 403
Sofa is placed in Hall.
Bed is in bedroom.
Hybrid Inheritance
Device is on
Charging battery
```

10. RESULT

Thus, the different types of inheritance in JavaScript — single, multilevel, hierarchical, multiple (using object mixins with Object.assign()), and hybrid inheritance — were successfully implemented using ES6 class syntax. Each type was verified by creating objects and confirming that the correct set of inherited and own methods executed as expected.